

SENIOR QA ENGINEER - REACT NATIVE, WEB & API AUTOMATION TECHNICAL ASSESSMENT

Automated Quality Engineering for CMS-Driven Mobile and Web Experiences

OVERVIEW

Design and implement a production-minded automated quality solution for React Native and React applications powered by a public Payload headless CMS.

This is a coding assessment. We are looking for a Senior QA Engineer who develops test software, understands React and React Native behavior, and can build reliable automation across API, component/integration, and mobile end-to-end layers.

This is not a manual testing exercise. A submission centered on manual test cases, checklists, recorded clicking, or a Postman collection without executable assertions will not meet the objective.

All code, documentation, test names, defect reports, commit messages, and the review conversation must be in English.

PUBLISHED CMS API

Default base URL:

<https://payload-cms-poc-seven.vercel.app>

Interactive API documentation:

<https://payload-cms-poc-seven.vercel.app/api/docs>

OpenAPI 3.1 contract:

<https://payload-cms-poc-seven.vercel.app/api/openapi.json>

Public content API:

<https://payload-cms-poc-seven.vercel.app/api/content/v1>

The mobile content contract version is 1.1; public operations require no authentication.

Requests require four query parameters: platform (ios/android), market (MX), audience (guest), and appVersion (semantic version).

Relevant operations:

- GET /api/content/v1/bootstrap
- GET /api/content/v1/pages/home
- GET /api/content/v1/pages/menu
- GET /api/content/v1/legal/privacy_policy
- GET /api/media/file/{filename}
- POST /api/form-submissions

Responses use a versioned envelope containing contractVersion, data, and optional metadata.

The shared CMS environment requires read-only testing. Do not submit spam forms; intercept or mock submission flows unless a test form is provided.

Mobile E2E Testing & Native Application Target

<https://github.com/luism-sundevs/payload-mobile-consumer>

Use this codebase for React Native UI, orchestration, and E2E scripts. It includes CMS connections, Jest, and React Native Testing Library. Baseline scenarios are reference context and do not count as candidate contributions.

Web E2E Testing Target

<https://payload-website-consumer.vercel.app/>

The Casa Maíz web deployment is a mandatory target. Playwright scripts must live in a detached repository separate from mobile. Sharing CMS contracts or mock data across projects is encouraged.

Submission Instructions & Constraints

Publish a private copy or establish an assessment/<candidate-name> branch for a distinct Pull Request instead of merging to main. Use a detached repository for web automation and API tests. Do not alter seed configurations, database entities, or CMS settings. Minimal application code modifications are allowed only to improve test infrastructure and must be explicitly explained.

WHAT WE WANT TO ASSESS

- Strong automation engineering and TypeScript skills
- Proven understanding of React and React Native rendering, effects, state, and component composition

- Ability to choose the correct test layer instead of forcing every assertion through end-to-end automation
- API contract, targeting, and client-integration validation
- Reliable mobile and browser automation with a practical iOS/Android strategy
- Test architecture, debugging, local reproducibility, and flakiness prevention
- Risk-based judgment supported by executable evidence
- Clear professional English

REQUIRED TOOLING

- TypeScript for API, utilities, fixtures, and component/integration automation
- Jest and React Native Testing Library for React Native component and integration tests
- One mobile end-to-end framework (Maestro, Detox, or Appium) and Playwright for browser automation
- A Node package manager of your choice

You may add focused supporting libraries, such as a runtime schema validator, OpenAPI client generator, network mock, or test reporter. Explain important dependency choices in the README.

Do not use low-code or record-and-playback tools as the primary solution. If Maestro is selected, the surrounding API, fixture, assertion, and reporting architecture must still demonstrate meaningful software engineering.

THE CHALLENGE

Build an automated quality solution across two coordinated codebases: extend the supplied mobile repository with React Native component, integration, and mobile end-to-end tests; and create a separate automation repository for the public CMS API/contract tests and Playwright tests against the React web application.

Both repositories must run locally with clear, independent commands. The mobile Pull Request must make candidate-authored tests and any justified production refactors easy to distinguish from the baseline. The separate automation repository must clearly identify its API and web test architecture, configuration, and trade-offs. Do not duplicate the mobile application into the automation repository.

PART 1 - QUALITY STRATEGY AS CODE

Before adding tests, define a concise risk model in the README.

Identify the highest risks across:

- Versioned content-contract compatibility

- Platform-specific delivery context
- React Native component composition and asynchronous behavior
- CMS-driven navigation, actions, and destination resolution
- Dynamic, optional, malformed, or future content
- Network failures, cache validity, offline fallback, and nextChangeAt
- Accessibility, large text, and touch interaction
- iOS and Android behavioral differences
- Mobile end-to-end reliability and diagnostics

Map each selected risk to the most appropriate automation layer: API, React Native component/integration, mobile end-to-end, or Playwright browser end-to-end.

Keep this section practical. We want to understand why each automated check exists, not receive a generic QA textbook or a manual regression plan.

PART 2 - API AND CONTRACT AUTOMATION

In the separate API and web automation repository, build executable API tests against the published Payload content API. The base URL and delivery context must be configurable, and tests must not depend on execution order.

At minimum, automate the following:

1. Contract envelope and runtime shape

Validate that successful responses contain supported contractVersion 1.1, data with the expected high-level shape, and valid optional metadata. Demonstrate how the suite detects an incompatible contract without rejecting harmless additive fields.

2. Bootstrap behavior

Validate GET /bootstrap for both ios and android contexts. Assert the structural integrity of navigation, promotions, feature flags, operational controls, alerts, and their optional states without coupling tests to irrelevant database IDs or exact editorial copy.

3. Dynamic page and legal delivery

Validate Home, Menu, and Privacy responses. Confirm that page metadata is coherent, layouts are renderable, every delivered block exposes a usable blockType, and platform targeting is respected.

4. Context and input validation

Validate the published OpenAPI constraints for platform, market, audience, and appVersion, then compare them with runtime behavior. If the implementation differs from the contract, preserve executable evidence, report the discrepancy as a defect, and describe an explicit known-failure handling policy that preserves visibility without weakening the assertion. Do not

weaken an assertion merely to obtain a green result. Assert that resolvedContext reflects valid requests.

5. Targeting, scheduling, and cache boundaries

Add an assertion around platform, audience, version, or visibility targeting. When nextChangeAt is present, verify its format and explain how a client should prevent targeted cached content from being treated as valid beyond that boundary.

6. Errors and media

Validate one safe 404 or malformed-request path and the supported error payload shapes. Verify that the client contract can handle both absolute media URLs and relative Payload media paths.

API tests must provide useful diagnostics, avoid fragile exact-content assertions, and never mutate the shared CMS.

PART 3 - REACT NATIVE COMPONENT AND INTEGRATION TESTING

In the supplied mobile repository, demonstrate real React experience using React Native Testing Library and Jest. The repository already includes baseline coverage; identify it in the README and add or materially improve tests at high-risk production boundaries such as:

- CMS block registry or renderer
- Bootstrap controller or application configuration provider
- Alert or operational-control presentation
- CMS link and destination resolver
- Page composition and navigation integration
- Cache, offline, or stale-content state
- Image and media URL resolver

At minimum, cover:

1. Dynamic rendering

Prove that a supported CMS block produces the expected accessible native UI and receives the relevant content without hardcoding the current payload order.

2. Forward compatibility

Prove that an unknown block type, missing optional field, empty array, or unavailable image is handled without crashing the React tree.

3. User interaction and navigation

Test a meaningful behavior such as dismissing an alert, selecting a CMS-provided destination, opening Privacy, or triggering a retry. Assert user-observable behavior rather than private implementation details.

4. Asynchronous, lifecycle, or cache behavior

Test one timer, effect, request cancellation, stale-response, cache-expiration, retry, or offline scenario. Control time and asynchronous work deterministically; do not use fixed sleeps.

5. Platform behavior

Add at least one meaningful platform-aware assertion for ios and android. Mock Platform.OS only at the appropriate boundary and avoid duplicating the whole suite when behavior is shared.

Use semantic queries, accessibility roles, labels, and intentional test IDs. Avoid snapshots as the only assertion and avoid testing private component internals.

PART 4 - MOBILE END-TO-END AUTOMATION

In the supplied mobile repository, implement a small, high-value mobile end-to-end suite using Maestro, Detox, or Appium. At least one critical journey must run end to end on one platform. Provide a credible configuration and execution strategy for the second platform.

Cover the following journeys:

1. CMS-driven Home to Menu journey

Launch the application, verify meaningful Home content, navigate using CMS-driven controls, open Menu, and assert a user-visible outcome that corresponds to the content contract rather than a hardcoded database ID.

2. Privacy or operational-state journey

Reach Privacy through a CMS-provided destination, or validate a high-value bootstrap-driven state such as an alert, maintenance notice, or recommended update. Choose the path with the highest stable value in the supplied build.

3. Failure or degradation scenario

Automate one deterministic failure at the most reliable layer. Examples include an unavailable content request, delayed response, incompatible contract version, missing optional data, or offline launch with and without valid cache. Verify that the application fails safely and provides a useful recovery path.

The mobile suite must use stable accessibility identifiers or intentional test IDs, avoid fixed sleeps, collect useful failure artifacts, and remain independent of shared CMS mutations.

PART 5 - REACT WEB END-TO-END AUTOMATION

In the separate API and web automation repository, use Playwright to automate the published Casa Maíz web application. Keep the browser project isolated from the API project while reusing CMS contract schemas, fixtures, or builders where this improves clarity.

Cover a focused browser slice: load Home and assert meaningful CMS-driven content; dismiss the initial notice and navigate to Menu or Privacy using user-facing selectors; intercept a relevant request to create one deterministic degradation scenario and verify safe recovery; and add one executable accessibility assertion. Capture a Playwright trace and a screenshot or video on failure. Required coverage may target Chromium; explain how the design could extend to other browsers.

TEST AUTOMATION QUALITY REQUIREMENTS

The suite must:

- Use stable, user-facing selectors or intentional accessibility/test identifiers
- Avoid fixed sleeps and arbitrary global retries
- Avoid dependence on test execution order
- Use reusable fixtures, builders, or helpers only where they improve clarity
- Keep assertions close to business and user outcomes
- Separate live API checks from deterministic mocked integration scenarios
- Capture useful logs, screenshots, video, traces, or device output on failure as supported by the selected tools
- Support reproducible local configuration without committed secrets
- Produce a machine-readable test report
- Exit with a non-zero status when required tests fail
- Be repeatable without resetting or mutating the CMS database

Do not build an unnecessary page-object or screen-object hierarchy. We will evaluate whether abstractions improve maintenance and diagnosis.

ACCESSIBILITY AUTOMATION

Add executable accessibility-focused assertions to at least one critical React Native component or journey.

At minimum, prove two relevant behaviors, such as:

- Accessible names, roles, states, and hints
- Touch target or interaction semantics
- Focus behavior after navigation or modal presentation
- Large text or dynamic type resilience
- Error or status announcements
- Meaningful alternative text for CMS media

Document what the selected automation can and cannot prove. A short limitation statement is expected; a manual checklist is not required.

DEFECT OR QUALITY-RISK REPORT

Provide one concise report in English based on evidence discovered during the exercise.

Preferred outcome:

- A reproducible defect with an automated failing test

If no product defect is found:

- Report a concrete quality risk, explain its user or delivery impact, and provide an executable test that would detect a regression

The report must include:

- Title
- Severity and rationale
- Affected platform, layer, or user journey
- Preconditions
- Reproduction command
- Expected result
- Actual result
- Automated evidence
- Likely technical cause or investigation direction

Do not weaken a valid test to make an actual defect appear green. Mark known defects explicitly and explain how they should be handled in the documented local test commands and reports.

LOCAL EXECUTION AND REPRODUCIBILITY

Hosted automation workflows are not required for this assessment. The reviewer must be able to install and run each repository independently on a local machine using the documented commands.

The documented local execution must include:

- Dependency installation with lockfile enforcement in both repositories
- Type-checking
- React Native component and integration tests from the mobile repository
- Public CMS API tests and Playwright browser tests from the separate automation repository
- Machine-readable reports and failure artifacts

Provide an executable local mobile end-to-end command for one platform and a credible documented setup for the second platform. Explain device or simulator prerequisites and where failure artifacts are stored.

Neither repository needs to publish or deploy anything. All tests must remain read-only against the public CMS, and required failures must return a non-zero exit status.

ENGLISH COMMUNICATION REQUIREMENT

All submission material must be written in clear English:

- README
- Test names
- Code comments
- Commit messages
- Defect or risk report
- Architecture and trade-off notes

Be prepared to present the solution and answer technical questions in English during the review.

AI AND CODE-AGENT USE

The use of code agents and any AI tools needed to complete this assessment is allowed. Candidates remain responsible for the submitted solution and must be able to explain, modify, debug, and defend every part of it during the review.

Briefly document in the README which AI tools or agents were used and the most relevant ways they contributed. Evaluation is based on engineering judgment, test design, code quality, reliability, and understanding—not on whether AI was used.

NON-FUNCTIONAL EXPECTATIONS

- Secrets and credentials must not be committed
- Test data and logs must not expose sensitive information
- The suite must be understandable by another engineer without a walkthrough
- TypeScript types should model the boundaries that matter
- Runtime validation is encouraged where external JSON enters the test system
- Parallel execution must be safe or deliberately disabled with a documented reason
- Flaky-test prevention is more valuable than retries
- Dependencies and abstractions must remain proportionate to the 3–4 day delivery window

OUT OF SCOPE

- A manual regression checklist as the primary deliverable
- Recorded clicking or low-code automation as the primary solution
- Testing Payload Admin authoring workflows
- Changing CMS seed data or production logic to simplify assertions
- Sending form submissions to the shared public CMS
- Load testing the entire platform
- Full visual coverage of every CMS block and device size
- Building a custom test-management platform or deployment infrastructure
- Broad cross-browser and responsive-device coverage beyond the required Chromium journey

DELIVERABLES

Submit exactly two coordinated deliverables:

1. Mobile repository Pull Request

- Use a private fork or the branch assessment/<candidate-name> in the supplied payload-mobile-consumer repository.
- Keep all React Native component, integration, accessibility, and mobile end-to-end work in this repository.
- Add or materially improve Jest and React Native Testing Library coverage at high-risk production boundaries.
- Add the selected Maestro, Detox, or Appium mobile end-to-end suite and its configuration in the same repository.
- Update the README with setup, dependency installation, type-checking, component/integration test commands, mobile E2E commands, the second-platform strategy, and the location of reports and failure artifacts.
- Document the mobile risk model, test architecture, data strategy, trade-offs, and an inventory separating baseline, added, and modified work.
- Generate machine-readable mobile test reports through the documented commands.

2. Separate API and web automation repository

- Create a separate repository containing the executable TypeScript API/contract tests and Playwright browser tests.
- Keep the API and browser projects isolated while reusing contracts, schemas, fixtures, or builders where this improves clarity.
- Test the public Payload CMS API and the published Casa Maíz web consumer without mutating shared CMS data.
- Include a README with setup, configuration, dependency installation, type-checking, API test commands, Playwright commands, and the location of reports and failure artifacts.
- Document the automation risk model, architecture, data strategy, and important trade-offs.

- f. Generate machine-readable API and Playwright reports through the documented commands.

Requirements that apply to both deliverables

- Both repositories must install and run independently with reproducible local commands.
- Required test failures must return a non-zero exit status.
- Do not commit dependency folders, credentials, simulator data, downloadable application binaries, or unnecessary generated artifacts.
- Include one defect or quality-risk report; it may live in the relevant repository as long as its location is clearly documented.
- Document the use of code agents or AI tools as required in the AI and Code-Agent Use section.

DEFINITION OF DONE

A reviewer should be able to check out the mobile Pull Request and clone the separate API/web automation repository, configure the public targets, install each repository independently, and run documented commands that:

- Execute independent, read-only API checks against the published CMS
- Execute meaningful candidate-added or materially improved React Native component and integration tests from the mobile repository
- Run at least one critical mobile end-to-end journey from the mobile repository and the required Playwright browser journey from the separate automation repository
- Demonstrate a credible second-platform strategy
- Perform accessibility-focused automated assertions
- Produce readable and machine-readable results
- Fail clearly when a regression is introduced

The solution must demonstrate code-based quality engineering. Manual test documentation alone is not an acceptable submission.

REVIEW CONVERSATION

Be prepared to discuss the following in English:

- Why each test belongs at its chosen layer
- How you would test a new CMS block without overfitting to current content
- How you would detect a breaking content-contract change
- How React Native rendering, effects, navigation, and Platform.OS influenced component tests
- How you would investigate and eliminate a flaky emulator, simulator, or device test
- How you would test offline behavior and the nextChangeAt cache boundary deterministically

- How required checks are divided between the mobile repository and the separate API/web automation repository
- How the architecture reuses contracts and fixtures appropriately across mobile and web
- Which important risks remain untested and why

We are looking for a Senior QA Engineer who writes dependable software to test software, understands React and React Native products deeply, and improves delivery confidence through automation.